

2D Spawner Package

Unity	6000.0.30f1+
license	MIT
last commit	june
version	1.0.5

[Read in Persian/پارسی](#)

A powerful and flexible 2D Spawner package for Unity, designed to streamline the process of spawning 2D objects with advanced features like automated spawning, collision safety, and grid-based placement.



Table of Contents

- [Features](#)
- [Quick Start](#)
- [Installation](#)
- [How to Use](#)
 - [Basic Setup](#)
 - [Automated Spawning](#)
 - [Collision Safe Placement](#)
 - [Grid-based Placement](#)
 - [Object Pooling](#)
 - [Events](#)
- [API Reference](#)
 - [Spawner](#)
 - [Spawnable](#)
 - [Automation](#)
 - [Limitation](#)
 - [CollisionSafety](#)
 - [GridPlacement](#)
 - [Inspector Integration](#)
 - [Example Usage](#)
- [Contributions](#)

- [License](#)

Features

- **Automated Spawning:** Easily configure objects to spawn automatically with customizable intervals.
- **Limitation Controls:** Set limitations based on time or count for automatic spawning.
- **Collision Safe Placement:** Ensures spawned objects do not overlap with existing colliders, finding a free space if needed.
- **Grid-based Placement:** Effortlessly place objects in a predefined grid structure.
- **Event-Driven:** Utilize Unity Events for flexible callbacks on spawn start, spawn end, and limitation reached.
- **Highly Customizable:** Exposes key parameters in the Inspector for easy setup without coding.
- **Object Pooling:** Utilize a highly efficient object pool to reuse spawned objects, drastically reducing instantiation overhead and improving performance.
- **Prefab Selection Control:** Easily switch between randomly choosing prefabs from the list or spawning them in a sequential order.

Installation

Prerequisites

This package relies on [UniTask](#) for asynchronous operations. Please ensure UniTask is installed in your Unity project **before** installing this 2D Spawner package.

First, add UniTask to your project from its GitHub repository.

2D Spawner installation

Install via Git URL using Unity Package Manager

To install `2D Spawner` via Git URL using Unity Package Manager, follow these steps:

1. Open your Unity project
2. Go to `Window > Package Manager`.
3. Click the `+` icon in the top-left corner of the package manager window.
4. Select `Add package from git URL ....`
5. Paste the following URL and click "Add":

`https://github.com/AbolfazlHo/Spawner.git?path=Assets/ir.soor.spawner2d`

Quick Start

1. Install `UniTask`.
2. Install the `2D Spawner` package via Git URL.
3. Create an empty `GameObject` (e.g., `MySpawner`) in your scene.
4. Add the `Spawner` component to it.
5. Assign the desired 2D prefabs to the `Spawnables` list. You can also toggle the `Choose Spawnable Randomly` property to switch between random and sequential prefab selection.
6. Add a spawn area `GameObject` (with a `Collider2D`) and assign it in the `Spawner` component.
7. Enable `Spawn Automatically`, or call `Spawn()` manually from script.

How to Use

1. Basic Setup

1. Create an empty `GameObject` in your scene (e.g., `MySpawner`).
2. Add the `Spawner` component to it.
3. Assign the 2D Prefabs you want to spawn to the `Spawnables` list.
4. Define a `Spawn Area GameObject` (e.g., a `Quad` with a `Collider2D`) to control where objects can spawn.

2. Automated Spawning

1. On the `Spawner` component, enable `Spawn Automatically`.
2. Expand the `Spawn Automation Settings` foldout.
3. Set the `Per Spawn Interval` (delay between spawns).
4. If `Stop Spawning Automatically` is enabled, configure `Limitation Settings`:
 - Choose `Limitation Type` (Count or Time).
 - Set `Limit Count By` (e.g., 10 objects) or `Limit Time By` (e.g., 60 seconds).

3. Collision Safe Placement

1. Enable `Is Collision Safe` on the `Spawner` component.
2. Expand `Collision Safety Settings`.
3. Ensure your `Spawnable` prefabs have appropriate `Collider2D` components. The system will try to find a free spot if the initial position is occupied.

4. Grid-based Placement

1. Within `Collision Safety Settings` , enable `Is Grid Placement` .
2. Expand `Grid Placement Settings` .
3. Set `Spawnable Size` (the base size of your spawned objects).
4. Adjust `Padding` to add space between grid cells.
5. Configure `Rows Count` or `Columns Count` to define your grid dimensions. The system will automatically calculate cell sizes.
 - If `Columns Count` is greater than 0, row and column are calculated based on it.
 - Otherwise, row and column are calculated based on Rows Count.

5. Object Pooling

1. In the `Spawner` component, enable `Use Object Pool` .
2. Expand `Pooler Settings` .
3. Don't Populate the `Objects to Pool` list. The component will handles it automatically.

6. Events

The `Automation` , and `Limitation` expose `UnityEvents` that you can hook into for custom logic:

- Automation:
 - `On Spawn Start Event` : Invoked when automatic spawning begins.
 - `On Spawn End Event` : Invoked when automatic spawning finishes (either naturally or when stopped).
- Limitation:
 - `On Spawn Start Event` : Invoked by `Automation` when its spawning starts (if automatic spawning is limited).
 - `On Limitation Reached Event` : Invoked when the defined spawn count or time limit is reached.
 - `On Spawn End Event` : Invoked by `Automation` when its spawning ends (if automatic spawning is limited).

This API Reference provides a complete overview of the structure, properties, and behavior of the Spawner2D package.

Spawner (MonoBehaviour)

Main class to control the spawning process of game objects in the scene.

Public Methods

`void Spawn()`

- Immediately spawns an object based on the current settings.

`void StopSpawning()`

- Stops the automatic spawning process (if active).

Inspector Properties

- `GameObject _spawnAreaGameObject`
 - `GameObject` that defines the spawn area. Typically contains a `BoxCollider2D`.
- `Spawnable[] _spawnables`
 - List of objects that can be spawned.
- `bool _spawnAutomatically`
 - Determines whether the spawn process starts automatically.
- `Automation _spawnAutomationSettings`
 - Settings related to automatic spawning.
- `bool _isCollisionSafe`
 - Enables collision safety checks during spawning.
- `CollisionSafety _collisionSafetySettings`
 - Settings related to collision safety (e.g., distance check or grid).
- `string _spawnableTag`
 - Custom tag for identifying spawnable objects.

- `bool _chooseSpawnableRandomly`
 - When true, a random prefab is chosen for each spawn; otherwise, prefabs are spawned sequentially from the `_spawnables` list.
 - `bool _useObjectPool`
 - Determines whether the spawner will use an object pool for efficient object management.
 - `Pooler.Pooler _poolerSettings`
 - Configuration settings for the object pool, including the prefabs to be pooled.
-

Spawnable

Marker script to indicate that an object is spawnable.

Automation

Controls the timing and conditions for starting/stopping automatic spawning.

Properties

- `float _perSpawnInterval`
 - Time interval between each spawn.
- `bool _stopSpawningAutomatically`
 - Determines whether the spawn process stops automatically.
- `Limitation _limitationSettings`
 - Limitations that stop the spawn process (by count or time).
- `UnityEvent _onSpawnStartEvent`
 - Event triggered at the start of spawning.
- `UnityEvent _onSpawnEndEvent`
 - Event triggered at the end of spawning.

Note: Only one of `limitationSettings` or `onSpawnStart/EndEvent` is used depending on `stopSpawningAutomatically`.

Limitation

Manages constraints for stopping the automatic spawning process.

Enum

```
LimitationType { Time, Count }
```

Properties

- `LimitationType _limitationType`
 - Type of limitation (Time or Count).
 - `float _limitTimeBy`
 - For Time: seconds after which spawning stops.
 - `int _limitCountBy`
 - For Count: number of spawns after which spawning stops.
 - `UnityEvent _onSpawnStartEvent`
 - `UnityEvent _onLimitationReachedEvent`
 - `UnityEvent _onSpawnEndEvent`
-

CollisionSafety

Collision safety settings to prevent overlapping spawned objects.

Properties

- `bool _isPlacement`
 - Enables basic distance-based collision safety.
- `bool _isGridPlacement`

- Enables grid-based placement.
 - `GridPlacement _gridPlacementSettings`
 - Settings for grid-based placement.
-

GridPlacement

Grid-based layout configuration.

Properties

- `Vector2 _spawnableSize`
 - Base size of each spawnable object.
 - `Vector2 _padding`
 - Additional space added between each cell.
 - `int _rowCount`
 - Number of rows. -1 for automatic calculation.
 - `int _columnCount`
 - Number of columns. -1 for automatic calculation.
-

Inspector Integration

Custom Editors

`PropertyDrawer` classes are implemented for `Automation`, `Limitation`, and `CollisionSafety` to enhance the user experience in the Inspector. Conditional display of properties is used based on settings.

Main Editor: `SpawnerInspector`

Custom Inspector renderer for `Spawner` with intuitive layout and grouped controls including:

- Spawn Settings
 - Automation
 - Collision Safety
 - Events
-

Example Usage


```
[SerializeField] private Spawner _spawner;

void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        _spawner.Spawn();
    }

    if (Input.GetKeyDown(KeyCode.LeftShift))
    {
        _spawner.StopSpawning();
    }
}
```

Let me know if you'd like more details about any class or internal behavior (e.g., UniTask usage).



Contributions

Contributions are welcome! If you find a bug or have a feature request, please open an issue on the [GitHub repository](#).



License

This package is distributed under the MIT License. See the [LICENSE](#) file for more information.